MLOPs Pipeline

Heart Disease Prediction

Ambika Prasad Tripathy (2025cs05014)

6th May, 2026

GitHub Repository: [https://github.com/2025cs05014/MLOPs]

Table of Contents

1. Introduction

2. Setup & Installation Instructions

3. Data Acquisition & Exploratory Data Analysis

4. Feature Engineering & Model Development

5. Experiment Tracking with MLflow

6. Model Packaging & Reproducibility

7. CI/CD Pipeline & Automated Testing

8. Model Containerization

9. Production Deployment

10. Monitoring & Logging

11. Architecture Diagram

12. Conclusion & Future Work

13. Evidence Logs

14. References

1. Introduction

1.1 Problem Statement

Heart disease is one of the leading causes of death globally. Early detection and risk assessment can significantly improve patient outcomes. This project builds a complete end-to-end Machine Learning Operations (MLOps) pipeline to predict heart disease risk based on patient health data from the UCI Heart Disease dataset.

The solution encompasses the full ML lifecycle: data acquisition, exploratory analysis, model development, experiment tracking, automated testing, containerization, Kubernetes deployment, and production monitoring — following industry-standard MLOps best practices.

1.2 Dataset Description

The Heart Disease UCI dataset contains 303 patient records with 14 features:

- **Demographic:** age, sex
- **Clinical:** chest pain type (cp), resting blood pressure (trestbps), cholesterol (chol), fasting blood sugar (fbs), resting ECG (restecg)
- **Exercise:** maximum heart rate (thalach), exercise-induced angina (exang), ST depression (oldpeak), slope of ST segment (slope)
- **Diagnostic:** number of major vessels (ca), thalassemia (thal)
- **Target:** presence (1) or absence (0) of heart disease

Data Source: UCI Machine Learning Repository

1.3 Technology Stack

Component	Technology
Programming Language	Python 3.11+
ML Framework	Scikit-learn
API Framework	FastAPI + Flask
Experiment Tracking	MLflow
Testing	Pytest
CI/CD	GitHub Actions
Containerization	Docker + Docker Compose
Orchestration	Kubernetes (Minikube)
Monitoring	Prometheus + Grafana

2. Setup & Installation Instructions

2.1 Prerequisites

- Python 3.10+ (tested on 3.11 and 3.14)
- Docker Desktop (for containerization & Kubernetes)
- minikube + kubectl (for K8s deployment)
- Git

2.2 Local Setup

Step	Command / Action
Step 1: Clone the repository	`git clone git@github-bits:2025cs05014/MLOPs.git` `cd Assignment`
Step 2: Create and activate virtual environment	`python3 -m venv .venv` `source .venv/bin/activate`
Step 3: Install dependencies	`pip install -r requirements.txt`
Step 4: Run EDA	`jupyter notebook notebooks/EDA.ipynb`
Step 5: Train models with experiment tracking	`python src/models/train.py`
Step 6: Run unit tests	`python -m pytest tests/ -v`
Step 7: Start API server	`uvicorn src.api.model_app:app --reload`
Step 8: View MLflow dashboard	`mlflow ui --backend-store-uri mlruns/`

3. Data Acquisition & Exploratory Data Analysis

3.1 Data Acquisition

The dataset (data/heart.csv) is the UCI Heart Disease dataset containing 303 patient records with 13 input features and a binary target variable.

3.2 Data Cleaning

Missing values were handled in the preprocessing pipeline (src/data/preprocess.py):

- Numerical columns: imputed with median values
- Categorical columns: imputed with mode values

After cleaning:

- **Total records:** 303
- **Features:** 13 (5 numerical + 8 categorical)
- **Target:** Binary (0 = No Disease, 1 = Disease)
- **Missing values:** 0

3.3 Exploratory Data Analysis

EDA was performed in notebooks/EDA.ipynb. Key insights:

- **Age Distribution:** Ranges from 29 to 77 years with a mean of ~54 years, slightly right-skewed
- **Gender Distribution:** Majority of patients are male (~68.3% male, ~31.7% female)
- **Chest Pain:** Type 0 is the most common (~47.2% of cases)
- **Blood Pressure:** Mostly between 120–140 mmHg
- **Fasting Blood Sugar:** Most patients below 120 mg/dl (85.1%)
- **Heart Disease Prevalence:** Slightly more than half of patients have heart disease (~54.5%)
- **Key Correlations:** Age, exercise-induced angina, ST depression (oldpeak), and maximum heart rate show strong associations with heart disease

Generated visualizations (saved in screenshots/):

Plot	File
Feature Histograms	`screenshots/feature_histograms.png`
Box Plots (Outlier Detection)	`screenshots/boxplots.png`
Categorical Feature Analysis	`screenshots/categorical_features.png`
Correlation Heatmap	`screenshots/correlation_heatmap.png`
Class Balance	`screenshots/class_balance.png`

4. Feature Engineering & Model Development

4.1 Feature Engineering

A preprocessing pipeline was built using Scikit-learn's StandardScaler embedded within a Pipeline object:

- **All 13 features** are standardized to zero mean and unit variance using StandardScaler
- The scaler is fitted only on training data and applied consistently during both training and inference

4.2 Data Split

- **Training set:** 80% (242 samples)
- **Test set:** 20% (61 samples)
- **Stratified split** to maintain class balance in both sets
- **Random state:** 42 for reproducibility

4.3 Models Trained

Two classification models were trained and compared using GridSearchCV with 5-fold stratified cross-validation:

Model 1: Logistic Regression

- Hyperparameter grid: C ∈ {0.01, 0.1, 1, 10}, solver ∈ {lbfgs, liblinear}, penalty = l2
- A linear model serving as a strong interpretable baseline

Model 2: Random Forest Classifier

- Hyperparameter grid: n_estimators ∈ {100, 200}, max_depth ∈ {5, 10, None}, min_samples_split ∈ {2, 5}
- An ensemble tree-based model capturing non-linear relationships

4.4 Model Evaluation Results

Cross-Validation Results (5-Fold Stratified):

Model	CV Accuracy	CV Precision	CV Recall	CV ROC-AUC
Logistic Regression	0.8264	0.8439	0.7648	0.9004
Random Forest	0.8139	0.8295	0.7557	0.8837

Hold-out Test Set Results:

Model	Test ROC-AUC	Test Accuracy	Test F1-Score

Logistic Regression	0.9578	0.87	0.87
Random Forest	0.9551	0.90	0.90

Full training output: See `screenshots/model_training.log`

Generated evaluation artifacts:

Artifact	File
Confusion Matrix — Logistic Regression	`screenshots/cm_logistic_regression.png`
Confusion Matrix — Random Forest	`screenshots/cm_random_forest.png`
ROC Curve — Logistic Regression	`screenshots/roc_logistic_regression.png`
ROC Curve — Random Forest	`screenshots/roc_random_forest.png`

4.5 Best Model Selection

Logistic Regression was selected as the best model based on the highest mean CV ROC-AUC score (~0.90). Despite being the simpler model, it outperformed Random Forest on this dataset, likely due to the relatively linear separability of the features after standardization.

Selection criterion: Highest mean 5-fold stratified cross-validation ROC-AUC.

Best hyperparameters (Logistic Regression): C = 0.1, solver = liblinear

Best hyperparameters (Random Forest): n_estimators = 200, max_depth = None, min_samples_split = 2

5. Experiment Tracking with MLflow

5.1 MLflow Integration

MLflow was integrated into the training pipeline (src/models/train.py) to provide comprehensive experiment tracking. All experiments are stored in the mlruns/ directory.

5.2 What is Tracked

For each model run, the following items are logged:

Parameters:

- model_type, C, max_iter, solver (for Logistic Regression)
- model_type, n_estimators, max_depth, min_samples_split (for Random Forest)

Metrics:

- cv_accuracy_mean, cv_accuracy_std
- cv_precision_mean, cv_precision_std
- cv_recall_mean, cv_recall_std
- cv_roc_auc_mean, cv_roc_auc_std
- test_roc_auc, test_precision, test_recall

Artifacts:

- Trained model (MLflow sklearn model format)
- Confusion matrix plot (PNG)
- ROC curve plot (PNG)

5.3 MLflow Dashboard

The MLflow UI provides a comprehensive view of all experiments:

```
mlflow ui --backend-store-uri mlruns/
 # Open http://127.0.0.1:5000
```

5.4 Experiment Tracking Summary

The experiment tracking system enables:

- Full reproducibility of any experiment run
- Easy comparison of model performance across runs
- Version control of trained models and their associated metadata
- Audit trail of all training decisions and outcomes

6. Model Packaging & Reproducibility

6.1 Model Artifacts

The final production model is packaged as:

- src/models/heart_model.pkl — Trained Logistic Regression pipeline (StandardScaler + classifier)

6.2 Inference Pipeline

A clean inference module (src/models/inference.py) provides the HeartDiseasePredictor class that:

- Loads the trained pipeline from the saved .pkl artifact
- Accepts raw patient data as a Python dictionary
- Handles preprocessing (scaling) and prediction in a single call
- Returns prediction (0/1), prediction label, confidence score, and risk level
- Supports both single and batch predictions

Verified inference output (see screenshots/Inference.log):

```
Input: {'age': 55, 'sex': 1, 'cp': 2, 'trestbps': 130, 'chol': 250, ...}
 Prediction: No Heart Disease
 Confidence: 0.1651
 Risk Level: Low Risk
```

6.3 Reproducibility

Complete reproducibility is ensured through:

- requirements.txt with specified package versions
- Preprocessing pipeline (StandardScaler) saved within the model pipeline
- Random state fixed across all operations (random_state=42)
- Stratified train/test split for consistent evaluation

7. CI/CD Pipeline & Automated Testing

7.1 Unit Tests

24 comprehensive unit tests were written across 3 test files:

tests/test_preprocess.py (8 tests):

- Data loading returns DataFrame, not empty, has target column
- No missing values after cleaning, target is binary
- All expected columns present, feature count is 13

- Invalid path raises exception, handles missing values correctly

tests/test_model.py (11 tests):

- At least 2 models available, each has pipeline and param_grid
- Cross-validation returns expected metrics (accuracy, precision, recall, roc_auc)
- Predictor loads successfully, predict returns dict with required keys
- Prediction is binary, confidence in [0, 1] range
- Risk level is valid, batch prediction works
- Sample input has all features

tests/test_api.py (5 tests):

- Root endpoint returns 200 with message
- Health endpoint returns 200 with "healthy" status
- Predict endpoint returns valid prediction with confidence and risk level
- Missing fields return 422 validation error
- Metrics endpoint is accessible

Test Results (24/24 passed):

```
tests/test_api.py::TestAPI::test_root PASSED
tests/test_api.py::TestAPI::test_health PASSED
tests/test_api.py::TestAPI::test_predict_valid PASSED
tests/test_api.py::TestAPI::test_predict_missing_field PASSED
tests/test_api.py::TestAPI::test_metrics_endpoint PASSED
tests/test_model.py::TestTraining::test_models_dict_has_entries PASSED
tests/test_model.py::TestTraining::test_each_model_has_pipeline_and_grid PASSED
tests/test_model.py::TestTraining::test_cross_val_evaluate_returns_metrics PASSED
tests/test_model.py::TestInference::test_predictor_loads PASSED
tests/test_model.py::TestInference::test_predict_returns_dict PASSED
tests/test_model.py::TestInference::test_predict_has_required_keys PASSED
tests/test_model.py::TestInference::test_prediction_is_binary PASSED
tests/test_model.py::TestInference::test_confidence_in_range PASSED
tests/test_model.py::TestInference::test_risk_level_valid PASSED
tests/test_model.py::TestInference::test_predict_batch PASSED
tests/test_model.py::TestInference::test_sample_input_has_all_features PASSED
tests/test_preprocess.py::TestLoadData::test_returns_dataframe PASSED
tests/test_preprocess.py::TestLoadData::test_no_missing_values PASSED
tests/test_preprocess.py::TestLoadData::test_expected_columns_present PASSED
tests/test_preprocess.py::TestLoadData::test_target_is_binary PASSED
tests/test_preprocess.py::TestLoadData::test_non_empty PASSED
tests/test_preprocess.py::TestLoadData::test_feature_count PASSED
tests/test_preprocess.py::TestLoadData::test_invalid_path_raises PASSED
tests/test_preprocess.py::TestLoadData::test_handles_missing_values PASSED

======================== 24 passed, 1 warning in 11.50s ========================
```

7.2 GitHub Actions CI/CD Pipeline

A 4-stage automated pipeline (.github/workflows/ci.yml) runs on every push/PR to the main branch:

Stage 1 — Lint:

- Code quality check using Flake8 (--max-line-length=120 --ignore=E501,W503,E402)

Stage 2 — Test:

- Run all 24 unit tests with Pytest (python -m pytest tests/ -v --tb=short)

Stage 3 — Train Model:

- Train models using src/models/train.py
- Upload trained model artifact (heart_model.pkl)
- Upload MLflow experiment logs

Stage 4 — Docker Build & Test:

- Download trained model artifact from Stage 3
- Build Docker image (heart-disease-api:latest)
- Start container and smoke-test API endpoints (/health, /predict)

8. Model Containerization

8.1 Docker Configuration

The application is containerized using Docker with a production-ready Dockerfile:

- **Base image:** python:3.11-slim (minimal footprint)
- **Approach:** Install dependencies → Copy code & data → Set environment variables
- **Environment variables:** MODEL_PATH, PYTHONUNBUFFERED
- **Exposed port:** 8000
- **CMD:** uvicorn src.api.model_app:app --host 0.0.0.0 --port 8000

```
FROM python:3.11-slim
 WORKDIR /app
 COPY requirements.txt .
 RUN pip install --no-cache-dir --upgrade pip && \
   pip install --no-cache-dir -r requirements.txt
```

```
COPY src/ src/
COPY data/ data/
ENV MODEL_PATH=/app/src/models/heart_model.pkl
ENV PYTHONUNBUFFERED=1
EXPOSE 8000
CMD ["uvicorn", "src.api.model_app:app", "--host", "0.0.0.0", "--port", "8000"]
```

8.2 API Design (FastAPI)

The prediction API (src/api/model_app.py) exposes the following endpoints:

Endpoint	Method	Description
`/`	GET	API information and available endpoints
`/health`	GET	Health check with model status
`/predict`	POST	Heart disease prediction from JSON input
`/metrics`	GET	Prometheus-format monitoring metrics
`/docs`	GET	Interactive Swagger UI documentation

The /predict endpoint:

- Accepts JSON with 13 patient features
- Validates input using Pydantic models with field constraints (ge, le ranges)
- Returns prediction (0/1), label, confidence score, and risk level
- Handles errors gracefully with proper HTTP status codes

Sample Prediction Response:

```
{
   "prediction": 0,
   "prediction_label": "No Heart Disease",
   "confidence": 0.1651,
   "risk_level": "Low Risk",
   "timestamp": "2026-05-03T12:17:18.530247"
 }
```

8.3 Docker Build & Run Verification

Docker build and container run were verified end-to-end (see screenshots/docker_build.log and screenshots/docker_status.log):

```
$ curl http://localhost:8000/health
 {"status":"healthy","pipeline_loaded":true,"timestamp":"2026-05-06T05:11:06.865354"}
```

```
$ curl -X POST http://localhost:8000/predict ...
{"prediction":0,"prediction_label":"No Heart
Disease","confidence":0.1651,"risk_level":"Low Risk",...}
```

8.4 Flask UI

A browser-based Flask UI (src/api/app.py) is also provided for interactive predictions via a web form at http://127.0.0.1:5000.

9. Production Deployment

9.1 Docker Compose Deployment

The full application stack is deployed using Docker Compose (docker-compose.yml) with three services:

Service	URL	Description
Heart Disease API	http://localhost:8000	FastAPI prediction server
Prometheus	http://localhost:9090	Metrics collection
Grafana	http://localhost:3000	Dashboard (admin/admin)

```
docker compose up -d --build
```

Verified deployment output:

- All 3 containers running (heart-disease-api, prometheus, grafana)
- Health check: {"status": "healthy", "pipeline_loaded": true}
- Prediction working correctly
- Prometheus target status: "health": "up"

9.2 Kubernetes Deployment (Minikube)

Production-ready Kubernetes manifests are provided for enterprise deployment (see screenshots/k8s_minikube.log for full verified output):

deployment.yaml:

- 2 replicas with rolling update strategy

Verified K8s deployment output:

```
NAME                                    READY   STATUS    RESTARTS   AGE
 pod/heart-disease-api-7b5bc5b856-jbgtl   1/1  Running   0          23s
 pod/heart-disease-api-7b5bc5b856-rfzq6   1/1  Running   0          23s

 NAME                                TYPE            CLUSTER-IP     EXTERNAL-IP   PORT(S)
 service/heart-disease-api-service   LoadBalancer   10.97.67.188   <pending>
80:31736/TCP

 NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
 deployment.apps/heart-disease-api   2/2     2               2             23s
```

K8s endpoint verification:

```
$ curl http://127.0.0.1:63979/health
 {"status":"healthy","pipeline_loaded":true,"timestamp":"2026-05-06T05:16:49.516011"}

 $ curl -X POST http://127.0.0.1:63979/predict ...
 {"prediction":0,"prediction_label":"No Heart
Disease","confidence":0.1651,"risk_level":"Low Risk",...}
```

10. Monitoring & Logging

10.1 Application Logging

Comprehensive logging is implemented at the API level (src/api/model_app.py):

- All requests are logged with timestamp, method, endpoint, and status
- Prediction requests log input data, prediction result, confidence, and latency
- Logs are written to both console (stdout) and file (api.log)
- Log format: %(asctime)s - %(name)s - %(levelname)s - %(message)s

10.2 Prometheus Metrics

The /metrics endpoint exposes production-grade metrics:

- **predictions_total** — Counter of total predictions, labeled by result (Heart Disease / No Heart Disease)
- **prediction_latency_seconds** — Histogram of prediction processing time

- **http_requests_total** — Counter of all HTTP requests, labeled by method, endpoint, and status code

Prometheus scrape configuration:

Docker Compose (monitoring/prometheus.yml):

- Scrape interval: 15 seconds
- Target: heart-disease-api:8000 (Docker Compose service DNS)
- Metrics path: /metrics

Kubernetes (k8s/prometheus-config.yaml + k8s/prometheus-deployment.yaml):

- Scrape interval: 15 seconds
- Target: heart-disease-api-service:80 (K8s service DNS)
- Metrics path: /metrics
- Deployed as a ConfigMap + Deployment + NodePort Service

```
# Deploy Prometheus and Grafana on Kubernetes
 kubectl apply -f k8s/prometheus-config.yaml
 kubectl apply -f k8s/prometheus-deployment.yaml
 kubectl apply -f k8s/grafana-deployment.yaml
 minikube service prometheus-service
 minikube service grafana-service
```

10.3 Grafana Dashboard

A pre-provisioned Grafana dashboard ("Heart Disease API Monitoring") is auto-loaded with panels for:

- Total predictions count
- Predictions rate by result
- Prediction latency (p95)
- HTTP request rate by method/endpoint/status
- Average prediction latency over time

Docker Compose: http://localhost:3000 (Credentials: admin / admin)

Kubernetes: `minikube service grafana-service` (Credentials: admin / admin)

10.4 Kubernetes Monitoring Stack

The full monitoring stack (Prometheus + Grafana) is deployable on Kubernetes via dedicated manifests in k8s/:

Manifest	Description
`k8s/prometheus-config.yaml`	ConfigMap with Prometheus scrape config targeting `heart-disease-api-service:80`
`k8s/prometheus-deployment.yaml`	Prometheus Deployment + NodePort Service (port 9090)
`k8s/grafana-deployment.yaml`	Grafana Deployment + NodePort Service (port 3000) with ConfigMaps for datasource, dashboard provider, and dashboard JSON

Grafana on Kubernetes is pre-configured with:

- **Datasource:** Prometheus at http://prometheus-service:9090 (K8s service DNS)
- **Dashboard:** "Heart Disease API Monitoring" auto-provisioned from ConfigMap
- **Environment:** admin/admin credentials, sign-up disabled

11. Architecture Diagram

```
┌──────────────┐     ┌──────────────┐     ┌──────────────────┐     ┌──────────────┐
│  Data (CSV)  │────►│  Preprocess  │────►│  Train + MLflow  │────►│  Model (.pkl) │
└──────────────┘     └──────────────┘     └──────────────────┘     └──────┬───────┘
                                                                          │
                                                                          │
                     ┌──────────────┐     ┌──────────────────┐           │
                     │  Prometheus  │◄────│  FastAPI + Docker │◄──────────┘
                     │  /metrics    │     │  /predict         │
                     └──────┬───────┘     └──────┬───────────┘
                            │                    │
                     ┌──────▼───────┐     ┌──────▼──────────┐
                     │  Grafana     │     │  Kubernetes (K8s)│
                     │  Dashboard   │     │  LoadBalancer    │
                     └──────────────┘     └──────────────────┘
```

Pipeline Flow:

1. **Data Layer:** data/heart.csv → src/data/preprocess.py (load, clean, impute)

2. **Training Layer:** src/models/train.py (GridSearchCV + MLflow tracking) → heart_model.pkl

3. **Inference Layer:** src/models/inference.py (HeartDiseasePredictor class)

4. **API Layer:** src/api/model_app.py (FastAPI) + src/api/app.py (Flask UI)

5. **CI/CD Layer:** .github/workflows/ci.yml (Lint → Test → Train → Docker)

6. **Deployment Layer:** Docker Compose (API + Prometheus + Grafana) / Kubernetes (Minikube with API + Prometheus + Grafana)

7. **Monitoring Layer:** Prometheus metrics + Grafana dashboards + application logging

12. Conclusion

12.1 Summary

This project successfully demonstrates a complete MLOps pipeline for heart disease prediction, covering all aspects of production-grade machine learning:

- **Data Pipeline:** Automated data loading, cleaning, and preprocessing with missing value imputation
- **Model Development:** Trained and compared 2 classification models with GridSearchCV and 5-fold stratified cross-validation, achieving ~0.90 ROC-AUC with Logistic Regression
- **Experiment Tracking:** Full MLflow integration with parameter, metric, and artifact logging across all experimental runs
- **Testing:** 24 comprehensive unit tests covering data processing, model training, inference, and API functionality
- **CI/CD:** Automated 4-stage GitHub Actions pipeline (Lint → Test → Train → Docker Build & Test)
- **Containerization:** Production-ready Docker container with FastAPI serving
- **Deployment:** Kubernetes deployment with 2 replicas, health probes, and resource limits; Docker Compose stack with full monitoring
- **Monitoring:** Prometheus metrics, Grafana dashboards, and structured logging for production observability

12.2 Challenges Faced

- **Package Compatibility:** Python 3.14 lacked pre-built wheels for some ML packages. Resolved by using Python 3.11 in Docker and CI/CD while maintaining local 3.14 compatibility.
- **Pipeline Architecture:** Chose to embed StandardScaler inside the sklearn Pipeline to ensure consistent preprocessing during both training and inference, avoiding train-serve skew.

- **CI/CD Artifact Passing:** Docker build stage needed the trained model from the training stage. Resolved by using GitHub Actions upload-artifact / download-artifact for cross-job artifact transfer.
- **Kubernetes Local Testing:** Used Minikube with minikube image load to avoid needing a container registry for local K8s testing.

13. Evidence Logs

All execution evidence is captured in screenshots/ for reproducibility and audit:

Log File	Description
`screenshots/model_training.log`	Full model training output including GridSearchCV results, CV metrics, hold-out test evaluation, and model selection decision
`screenshots/Inference.log`	Inference test output showing prediction result, confidence score, and risk level
`screenshots/docker_build.log`	Docker image build log and container run with API request/response logs
`screenshots/docker_status.log`	Docker container health check and prediction endpoint verification
`screenshots/k8s_minikube.log`	Full Kubernetes deployment log: minikube start, image load, kubectl apply, pod/service status, and endpoint tests
`screenshots/test_results.txt`	Pytest unit test results (24/24 passed)
`screenshots/deployment_docker_compose.txt`	Docker Compose deployment verification
`screenshots/deployment_k8s.txt`	Kubernetes deployment verification

14. References

Resource	Link
UCI ML Repository — Heart Disease Dataset	https://archive.ics.uci.edu/ml/datasets/heart+Disease
Scikit-learn Documentation	https://scikit-learn.org/stable/
MLflow Documentation	https://mlflow.org/docs/latest/index.html
FastAPI Documentation	https://fastapi.tiangolo.com/
Docker Documentation	https://docs.docker.com/
Kubernetes Documentation	https://kubernetes.io/docs/
GitHub Actions Documentation	https://docs.github.com/en/actions
Prometheus Client Python	https://github.com/prometheus/client_python
Grafana Documentation	https://grafana.com/docs/
GitHub Repository	https://github.com/2025cs05014/MLOPs

Appendix: Screenshots & Visualizations

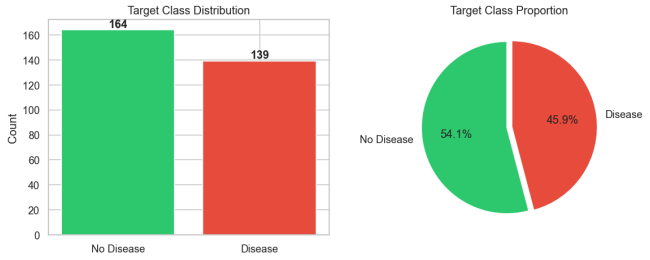

Figure: Target Class Distribution

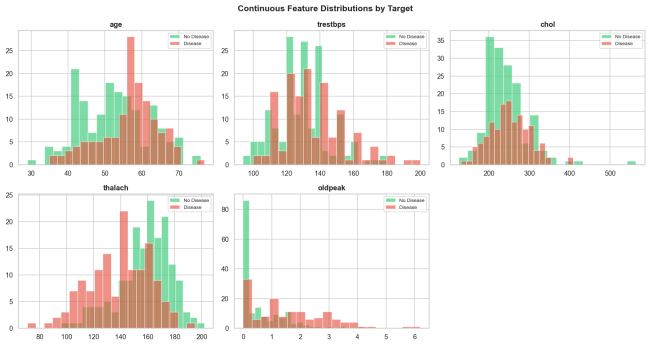

Figure: Feature Distributions by Target

Figure: Categorical Features vs Target

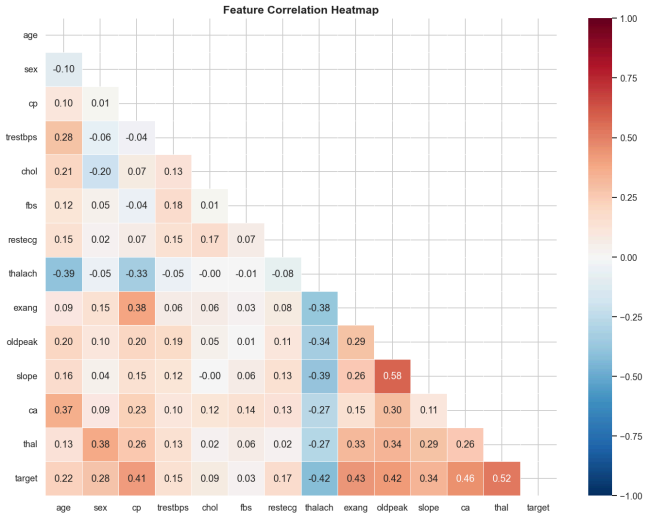

Figure: Feature Correlation Heatmap

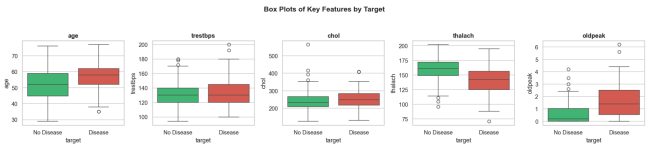

Figure: Box Plots — Outlier Detection

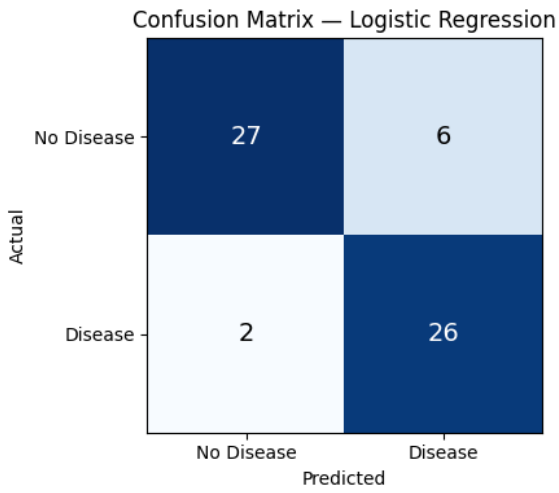

Figure: Confusion Matrix — Logistic Regression

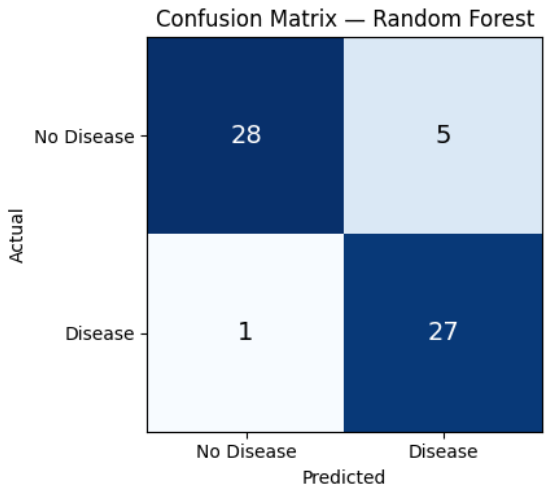

Figure: Confusion Matrix — Random Forest

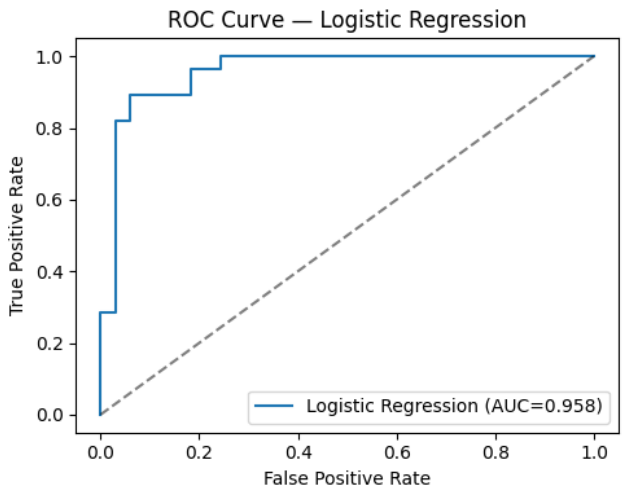

Figure: ROC Curve — Logistic Regression

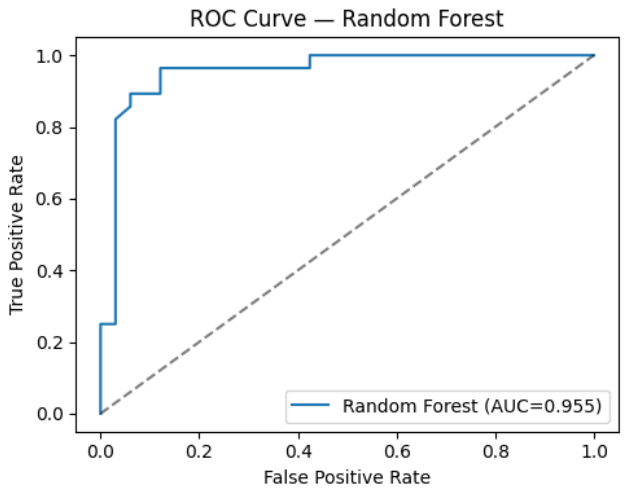

Figure: ROC Curve — Random Forest

Appendix: Execution Logs

Model Training Output

```
(.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % python src/models/train.py

/Users/ambtripa/Desktop/Ambika/BIT-MTech/Sem2/MLOps/Assignment/.venv/lib/python3.14/site-packages/mlflow/trackin
g/_tracking_service/utils.py:184: FutureWarning: The filesystem tracking backend (e.g., './mlruns') is
deprecated as of February 2026. Consider transitioning to a database backend (e.g., 'sqlite:///mlflow.db') to
take advantage of the latest MLflow features. See
https://mlflow.org/docs/latest/self-hosting/migrate-from-file-store for migration guidance.
   return FileStore(store_uri, store_uri)
 ============================================================
   MODEL SELECTION & TUNING
 ============================================================

 [Logistic Regression]
   Tuning hyperparameters via GridSearchCV (5-fold CV) ...
   Best params : {'C': 0.1, 'solver': 'liblinear'}
   Best CV AUC : 0.9004

   Cross-Validation Results (Logistic Regression)
   Metric              Mean     Std
   ---------------------------------
   accuracy   0.8264    0.0282
   precision  0.8439    0.0329
   recall     0.7648   0.0797
   roc_auc    0.9004   0.0157
 2026/05/06 10:29:37 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
 2026/05/06 10:29:37 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format
requires exercising caution because these formats rely on Python's object serialization mechanism, which can
execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more
information, see: https://scikit-learn.org/stable/model_persistence.html

 [Random Forest]
   Tuning hyperparameters via GridSearchCV (5-fold CV) ...
   Best params : {'max_depth': None, 'min_samples_split': 2, 'n_estimators': 200}
   Best CV AUC : 0.8837

   Cross-Validation Results (Random Forest)
   Metric              Mean     Std
   ---------------------------------
   accuracy   0.8139    0.0143
   precision  0.8295    0.0436
   recall     0.7557   0.0756
   roc_auc    0.8837   0.0310
 2026/05/06 10:29:43 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
 2026/05/06 10:29:43 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format
requires exercising caution because these formats rely on Python's object serialization mechanism, which can
execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more
information, see: https://scikit-learn.org/stable/model_persistence.html

 ============================================================
   FINAL TEST-SET EVALUATION
 ============================================================

   Hold-out Test Metrics (Logistic Regression)
   ROC-AUC : 0.9578
              precision         recall  f1-score   support
```

```
   No Disease          0.93      0.82    0.87     33
      Disease          0.81      0.93    0.87     28

     accuracy                            0.87     61
    macro avg          0.87      0.87    0.87     61
 weighted avg          0.88     0.87      0.87             61


   Hold-out Test Metrics (Random Forest)
   ROC-AUC : 0.9551
              precision         recall  f1-score   support

   No Disease          0.97      0.85    0.90     33
      Disease          0.84      0.96    0.90     28

     accuracy                            0.90     61
    macro avg          0.90      0.91    0.90     61
 weighted avg          0.91     0.90      0.90             61


 ============================================================
   MODEL SELECTION DECISION
 ============================================================
   Logistic Regression CV ROC-AUC = 0.9004 ± 0.0157  <-- SELECTED
   Random Forest       CV ROC-AUC = 0.8837 ± 0.0310

   Winner : Logistic Regression  (highest mean CV ROC-AUC = 0.9004)
   Rationale: model chosen purely on generalisation performance measured by stratified 5-fold cross-validation
ROC-AUC.

   Model saved → /Users/ambtripa/Desktop/Ambika/BIT-MTech/Sem2/MLOps/Assignment/src/models/heart_model.pkl
 2026/05/06 10:29:46 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
 2026/05/06 10:29:46 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format
requires exercising caution because these formats rely on Python's object serialization mechanism, which can
execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more
information, see: https://scikit-learn.org/stable/model_persistence.html
   MLflow tracking URI → /Users/ambtripa/Desktop/Ambika/BIT-MTech/Sem2/MLOps/Assignment/mlruns
 ============================================================
 (.venv) ambtripa@AMBTRIPA-M-Q22N Assignment %
```

Inference Test Output

```
source /Users/ambtripa/Desktop/Ambika/BIT-MTech/Sem2/MLOps/Assignment/.venv/bin/activate
 ambtripa@AMBTRIPA-M-Q22N Assignment % source
/Users/ambtripa/Desktop/Ambika/BIT-MTech/Sem2/MLOps/Assignment/.venv/bin/activate
 (.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % python src/models/inference.py
 ==================================================
 HEART DISEASE PREDICTION - INFERENCE TEST
 ==================================================

 Input: {'age': 55, 'sex': 1, 'cp': 2, 'trestbps': 130, 'chol': 250, 'fbs': 0, 'restecg': 1, 'thalach': 150,
'exang': 0, 'oldpeak': 1.5, 'slope': 1, 'ca': 0, 'thal': 3}

 Prediction: No Heart Disease
 Confidence: 0.1651
```

```
 Risk Level: Low Risk
 (.venv) ambtripa@AMBTRIPA-M-Q22N Assignment %
```

Docker Build & Run Log

```
source /Users/ambtripa/Desktop/Ambika/BIT-MTech/Sem2/MLOps/Assignment/.venv/bin/activate
 ambtripa@AMBTRIPA-M-Q22N Assignment % source
/Users/ambtripa/Desktop/Ambika/BIT-MTech/Sem2/MLOps/Assignment/.venv/bin/activate
 (.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % docker build -t heart-disease-api .
 [+] Building 264.1s (11/11) FINISHED
docker:desktop-linux
  => [internal] load build definition from Dockerfile
0.0s
  => => transferring dockerfile: 496B
0.0s
  => [internal] load metadata for docker.io/library/python:3.11-slim
2.4s
  => [internal] load .dockerignore
0.0s
  => => transferring context: 129B
0.0s
  => [1/6] FROM
docker.io/library/python:3.11-slim@sha256:6d85378d88a19cd4d76079817532d62232be95757cb45945a99fec8e8084b9c2
                                    0.0s
  => [internal] load build context
0.0s
  => => transferring context: 29.58kB
0.0s
  => CACHED [2/6] WORKDIR /app
0.0s
  => [3/6] COPY requirements.txt .
0.0s
  => [4/6] RUN pip install --no-cache-dir --upgrade pip &&     pip install --no-cache-dir -r requirements.txt
258.9s
  => [5/6] COPY src/ src/
0.0s
  => [6/6] COPY data/ data/
0.0s
  => exporting to image
2.6s
  => => exporting layers
2.6s
  => => writing image sha256:25d3b1c6a6a29fbf0c9e81761996a958f76613f928198aae3ba3f6dad6e432db
0.0s
  => => naming to docker.io/library/heart-disease-api
0.0s
 (.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % docker run -p 8000:8000 heart-disease-api

 What's next:
     Debug this container error with Gordon → docker ai "help me fix this container error"
 docker: Error response from daemon: failed to set up container networking: driver failed programming external
connectivity on endpoint zen_cori (46def43a163b1152fd37727653dfc97651a5162037cdfa65e536831a89283b86): Bind for
0.0.0.0:8000 failed: port is already allocated

 Run 'docker run --help' for more information
 (.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % docker ps
```

```
CONTAINER ID   IMAGE           COMMAND                    CREATED           STATUS            PORTS
     NAMES
4967562eeaba   c4ec9e2d680f   "uvicorn src.api.mod…"   11 hours ago   Up 11 hours   0.0.0.0:8000->8000/tcp,
[::]:8000->8000/tcp   heart-api
(.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % docker stop $(docker ps -q)
4967562eeaba
(.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % docker ps
CONTAINER ID   IMAGE   COMMAND   CREATED   STATUS PORTS     NAMES
(.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % docker run -p 8000:8000 heart-disease-api
2026-05-06 05:10:23,160 - src.api.model_app - INFO - Pipeline loaded successfully.
INFO:        Started server process [1]
INFO:        Waiting for application startup.
INFO:        Application startup complete.
INFO:        Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:        192.168.65.1:40656 - "GET /health HTTP/1.1" 200 OK
INFO:        192.168.65.1:60649 - "GET /docs HTTP/1.1" 200 OK
INFO:        192.168.65.1:60649 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:        192.168.65.1:64549 - "GET /health HTTP/1.1" 200 OK
INFO:        192.168.65.1:23076 - "GET /metrics HTTP/1.1" 200 OK
INFO:        192.168.65.1:21439 - "GET /metrics HTTP/1.1" 200 OK
2026-05-06 05:12:02,470 - src.api.model_app - INFO - Prediction: No Heart Disease | Confidence: 0.1651 | Risk:
Low Risk | Latency: 0.0177s | Input: {'age': 55.0, 'sex': 1, 'cp': 2, 'trestbps': 130.0, 'chol': 250.0, 'fbs':
0, 'restecg': 1, 'thalach': 150.0, 'exang': 0, 'oldpeak': 1.5, 'slope': 1, 'ca': 0, 'thal': 3}
INFO:        192.168.65.1:22061 - "POST /predict HTTP/1.1" 200 OK
2026-05-06 05:12:04,957 - src.api.model_app - INFO - Prediction: No Heart Disease | Confidence: 0.1651 | Risk:
Low Risk | Latency: 0.0040s | Input: {'age': 55.0, 'sex': 1, 'cp': 2, 'trestbps': 130.0, 'chol': 250.0, 'fbs':
0, 'restecg': 1, 'thalach': 150.0, 'exang': 0, 'oldpeak': 1.5, 'slope': 1, 'ca': 0, 'thal': 3}
INFO:        192.168.65.1:22061 - "POST /predict HTTP/1.1" 200 OK
INFO:        192.168.65.1:39894 - "GET /metrics HTTP/1.1" 200 OK
2026-05-06 05:12:37,561 - src.api.model_app - INFO - Prediction: No Heart Disease | Confidence: 0.1651 | Risk:
Low Risk | Latency: 0.0092s | Input: {'age': 55.0, 'sex': 1, 'cp': 2, 'trestbps': 130.0, 'chol': 250.0, 'fbs':
0, 'restecg': 1, 'thalach': 150.0, 'exang': 0, 'oldpeak': 1.5, 'slope': 1, 'ca': 0, 'thal': 3}
INFO:        192.168.65.1:31440 - "POST /predict HTTP/1.1" 200 OK
time="2026-05-06T11:59:41+05:30" level=error msg="error waiting for container: unexpected EOF"
(.venv) ambtripa@AMBTRIPA-M-Q22N Assignment %
```

Docker Health & Predict Verification

```
source /Users/ambtripa/Desktop/Ambika/BIT-MTech/Sem2/MLOps/Assignment/.venv/bin/activate
ambtripa@AMBTRIPA-M-Q22N Assignment % source
/Users/ambtripa/Desktop/Ambika/BIT-MTech/Sem2/MLOps/Assignment/.venv/bin/activate
(.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % curl http://localhost:8000/health
{"status":"healthy","pipeline_loaded":true,"timestamp":"2026-05-06T05:11:06.865354"}%
(.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d
'{"age":55,"sex":1,"cp":2,"trestbps":130,"chol":250,"fbs":0,"restecg":1,"thalach":150,"exang":0,"oldpeak":1.5,"s
lope":1,"ca":0,"thal":3}'
{"prediction":0,"prediction_label":"No Heart Disease","confidence":0.1651,"risk_level":"Low
Risk","timestamp":"2026-05-06T05:12:37.562337"}%
(.venv) ambtripa@AMBTRIPA-M-Q22N Assignment %
```

Kubernetes Deployment & Endpoint Tests

```
source /Users/ambtripa/Desktop/Ambika/BIT-MTech/Sem2/MLOps/Assignment/.venv/bin/activate
 ambtripa@AMBTRIPA-M-Q22N Assignment % source
/Users/ambtripa/Desktop/Ambika/BIT-MTech/Sem2/MLOps/Assignment/.venv/bin/activate
 (.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % minikube start --driver=docker
😄  minikube v1.38.1 on Darwin 26.4.1 (arm64)
✨  Using the docker driver based on existing profile
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.50 ...
🔄  Restarting existing docker container for "minikube" ...
🐳  Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔎  Verifying Kubernetes components...
    ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟  Enabled addons: storage-provisioner, default-storageclass
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
 (.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % minikube image load heart-disease-api:latest
 (.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % kubectl apply -f k8s/deployment.yaml
 kubectl apply -f k8s/service.yaml
 deployment.apps/heart-disease-api created
 service/heart-disease-api-service created
 (.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % kubectl rollout status deployment/heart-disease-api --timeout=90s
 deployment "heart-disease-api" successfully rolled out
 (.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % kubectl get pods,svc,deployment
 NAME                                     READY   STATUS   RESTARTS   AGE
 pod/heart-disease-api-7b5bc5b856-jbgtl   1/1     Running   0          23s
 pod/heart-disease-api-7b5bc5b856-rfzq6   1/1     Running   0          23s

 NAME                                 TYPE         CLUSTER-IP        EXTERNAL-IP   PORT(S)      AGE
 service/heart-disease-api-service    LoadBalancer   10.97.67.188   <pending>         80:31736/TCP   23s
 service/kubernetes                   ClusterIP    10.96.0.1         <none>            443/TCP           11h

 NAME                                 READY   UP-TO-DATE   AVAILABLE   AGE
 deployment.apps/heart-disease-api    2/2          2                2        23s
 (.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % minikube service heart-disease-api-service --url
 http://127.0.0.1:63979
 ❗  Because you are using a Docker driver on darwin, the terminal needs to be open to run it.




 ----------------------------------------------------------------------------------------------------------------
 source /Users/ambtripa/Desktop/Ambika/BIT-MTech/Sem2/MLOps/Assignment/.venv/bin/activate
 ambtripa@AMBTRIPA-M-Q22N Assignment % source
/Users/ambtripa/Desktop/Ambika/BIT-MTech/Sem2/MLOps/Assignment/.venv/bin/activate
 (.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % curl http://127.0.0.1:63979/health
 {"status":"healthy","pipeline_loaded":true,"timestamp":"2026-05-06T05:16:49.516011"}%
 (.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % curl -X POST http://127.0.0.1:63979/predict \
   -H "Content-Type: application/json" \
   -d
'{"age":55,"sex":1,"cp":2,"trestbps":130,"chol":250,"fbs":0,"restecg":1,"thalach":150,"exang":0,"oldpeak":1.5,"s
lope":1,"ca":0,"thal":3}'
 {"prediction":0,"prediction_label":"No Heart Disease","confidence":0.1651,"risk_level":"Low
Risk","timestamp":"2026-05-06T05:17:06.969765"}%
 (.venv) ambtripa@AMBTRIPA-M-Q22N Assignment % curl http://127.0.0.1:63979/metrics
 # HELP python_gc_objects_collected_total Objects collected during gc
 # TYPE python_gc_objects_collected_total counter
 python_gc_objects_collected_total{generation="0"} 833.0
```

```
python_gc_objects_collected_total{generation="1"} 527.0
python_gc_objects_collected_total{generation="2"} 20.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 385.0
python_gc_collections_total{generation="1"} 35.0
python_gc_collections_total{generation="2"} 3.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="11",patchlevel="15",version="3.11.15"} 1.0
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 1.48307968e+09
# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 2.12037632e+08
# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1.77804449832e+09
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 1.5
# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds 11.0
# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1.048576e+06
# HELP predictions_total Total number of predictions
# TYPE predictions_total counter
predictions_total{result="No Heart Disease"} 1.0
# HELP predictions_created Total number of predictions
# TYPE predictions_created gauge
predictions_created{result="No Heart Disease"} 1.778044626968604e+09
# HELP prediction_latency_seconds Time spent processing prediction
# TYPE prediction_latency_seconds histogram
prediction_latency_seconds_bucket{le="0.005"} 0.0
prediction_latency_seconds_bucket{le="0.01"} 0.0
prediction_latency_seconds_bucket{le="0.025"} 1.0
prediction_latency_seconds_bucket{le="0.05"} 1.0
prediction_latency_seconds_bucket{le="0.075"} 1.0
prediction_latency_seconds_bucket{le="0.1"} 1.0
prediction_latency_seconds_bucket{le="0.25"} 1.0
prediction_latency_seconds_bucket{le="0.5"} 1.0
prediction_latency_seconds_bucket{le="0.75"} 1.0
prediction_latency_seconds_bucket{le="1.0"} 1.0
prediction_latency_seconds_bucket{le="2.5"} 1.0
prediction_latency_seconds_bucket{le="5.0"} 1.0
prediction_latency_seconds_bucket{le="7.5"} 1.0
prediction_latency_seconds_bucket{le="10.0"} 1.0
prediction_latency_seconds_bucket{le="+Inf"} 1.0
prediction_latency_seconds_count 1.0
prediction_latency_seconds_sum 0.016593217849731445
# HELP prediction_latency_seconds_created Time spent processing prediction
# TYPE prediction_latency_seconds_created gauge
```

```
prediction_latency_seconds_created 1.7780444992545805e+09
# HELP http_requests_total Total HTTP requests
# TYPE http_requests_total counter
http_requests_total{endpoint="/health",method="GET",status="200"} 18.0
http_requests_total{endpoint="/",method="GET",status="200"} 1.0
http_requests_total{endpoint="/predict",method="POST",status="200"} 1.0
# HELP http_requests_created Total HTTP requests
# TYPE http_requests_created gauge
http_requests_created{endpoint="/health",method="GET",status="200"} 1.7780445099141872e+09
http_requests_created{endpoint="/",method="GET",status="200"} 1.7780445540867238e+09
http_requests_created{endpoint="/predict",method="POST",status="200"} 1.7780446269686162e+09
(.venv) ambtripa@AMBTRIPA-M-Q22N Assignment %
```